Energy Consumption Prediction System

Project Id: 23

Team Id:PY-PT17

Name : Saksham Jain (Team Leader)
Class Roll No.:17
University Roll No.: 2028479
Section :P(G1)

Name : Satyam Kumar
Class Roll No.:22
University Roll No.:2028488
Section :P(G1)

Name : Ayush Kumar Verma
Class Roll No.:36
University Roll No.: 2028684
Section :P(G1)

TABLE OF CONTENTS

1. Problem Description

The Energy Consumption Prediction System is a comprehensive machine learning pipeline designed to estimate the electrical power usage of various institutional buildings based on operational, temporal, and environmental factors. Energy consumption forecasting is a critical component of modern facility management, enabling institutions to optimize HVAC systems, reduce carbon footprints, and achieve significant financial savings.

Previously, a baseline model was developed for this task, but it suffered from severe performance issues, yielding an R-squared (R^2) score of only 0.37. The underlying issues were deeply rooted in suboptimal data preprocessing and feature selection strategies.

Specifically, the original approach utilized a rudimentary dropna()operation to handle missing values, which catastrophically resulted in the loss of 1,069 rows—constituting 51% of the entire training dataset. Furthermore, the previous model relied on an overly aggressive forward selection method that stripped the dataset down to merely 8 features, ignoring highly predictive temporal and environmental signals.

Lastly, the target variable contained erroneous negative values and extreme outliers (up to 2,749 kWh) that severely skewed the model's loss gradients, while relying on default, un-tuned hyperparameters.

1.1 Dataset Description

The dataset consists of multivariate records detailing the operational status of campus buildings. All 21 original features were preserved, supplemented by 4 engineered interaction terms.

Feature Name	Description	Data Type
Time_of_Day	Categorical period (Morning, Afternoon, etc.)	Encoded
Students_Present	Number of individuals in the facility	Numeric
Temperature_C	Ambient exterior temperature in Celsius	Numeric
Appliance_Usage	Baseline power draw from plugged-in devices	Numeric
Season	Categorical season (Fall, Spring, Winter, Summer)	Encoded
Building_Type	Specific facility (Auditorium, Lab, Library, etc.)	Encoded
Weather_Condition	Sunny, Cloudy, Rainy, Stormy, etc.	Encoded
Occupancy_Level	High, Medium, Low categories	Encoded
HVAC_Mode	Heating, Cooling, Fan-only, Off	Encoded
Event_Type	Routine vs. Special Event tagging	Encoded
Maintenance_Status	Excellent, Fair, Poor maintenance grades	Encoded
Humidity_Percent	Exterior relative humidity	Numeric

Feature Name	Description	Data Type
Solar_Radiation_Wm2	Solar irradiance affecting building thermal mass	Numeric
HVAC_Usage_kW	Power consumption of climate control	Numeric
CO2_Level_ppm	Internal air quality & occupancy metric	Numeric
Energy_kWh	**Target Variable: Total Energy Consumed**	**Numeric**

1.2 Flow Diagram (Logical Architecture)

The architecture of the system follows a sequential data pipeline, ensuring seamless transitions from raw data ingestion to interactive user predictions.

1. **Data Collection & Ingestion:** Reading the raw operational dataset.
2. **Preprocessing & Cleaning:**
    - Removing negative values in the target variable.
    - Removing extreme target outliers using IQR percentile bands.
    - Applying median imputation to salvage 51% of the dataset previously lost to `dropna()`.
3. **Feature Engineering:** Generating polynomial/interaction features (e.g., ***CO2_Level* × *Students***) and encoding categorical variables using `LabelEncoder`. Scaling numerical dimensions via `StandardScaler`.
4. **Model Search & Training:** Splitting data into training and testing sets, followed by exhaustive `GridSearchCV` across multiple algorithms.
5. **Artifact Serialization:** Exporting the best model, scalers, encoders, and metadata into `.pkl` files.
6. **Interactive Prediction System:** Loading artifacts into `predict.ipynb`, accepting terminal user inputs, dynamically computing interactions, and logging results.

2. Modules for Project Implementation

2.1 Modules Used

The project heavily relies on the Python data science ecosystem to facilitate efficient data manipulation, mathematical operations, and robust machine learning implementations.

- **Pandas (`pandas`):** Used as the primary data manipulation engine. Functions such as `pd.read_csv()` were utilized to load historical data. Methods like `fillna()`, `apply()`, and `drop()` were critical for the median imputation and outlier removal phases. DataFrame structures form the backbone of the inference pipeline.

- **NumPy (`numpy`):** Provided support for high-performance vectorized numerical operations. NumPy arrays were frequently used under the hood by Scikit-Learn, and functions like `np.median()` and `np.percentile()` were essential for custom outlier detection bounds.

- **Scikit-Learn (`sklearn`):** The core machine learning library utilized in this architecture.
    - *Preprocessing:* `StandardScaler` normalized inputs to a mean of 0 and variance of 1. `LabelEncoder` mapped categorical strings to integer arrays.
    - *Ensemble Models:* Provided the `GradientBoostingRegressor` class.
    - *Model Selection:* `train_test_split` and `GridSearchCV`.

- **Joblib (`joblib`):** Acted as the serialization module. Once the optimal model was found, `joblib.dump()` was used to export the model weights (`best_model.pkl`) to disk.

2.2 Platform Used

Jupyter Notebook: Jupyter was selected as the primary integrated development environment. The interactive REPL (Read-Eval-Print Loop) nature of Jupyter cells is uniquely suited for machine learning workflows. It allowed the development team to iteratively test data preprocessing methods, visualize outlier distributions inline, and maintain the memory state of heavy machine learning objects. The project is explicitly divided into modular `.ipynb` files: `Model_Improve+Export.ipynb` for batch training and `predict.ipynb` for the end-user.

3. Machine Learning Model / Searching Technique Used

During the improvement phase, a wide array of regression algorithms were evaluated to find the most suitable mathematical mapping. The models tested included Random Forests, Support Vector Regressors (SVR), Ridge Regression, K-Nearest Neighbors, and Decision Trees.

However, the definitive best-performing model, serialized as `best_model.pkl`, was the **Gradient Boosting Regressor**.

Gradient Boosting is a powerful ensemble machine learning algorithm that builds a strong predictive model by sequentially combining the outputs of multiple weak learners—typically shallow decision trees. Unlike Random Forests, which build trees independently, Gradient Boosting builds trees in a forward stage-wise fashion,

where each new tree is explicitly trained to correct the residual errors of the accumulated ensemble.

Gradient Boosting Mathematics

Mathematically, the model initializes with a constant value $F_0(x)$, typically the mean of the target variable. For each iteration $m = 1$ to M (where M is `n_estimators`):

1. Compute the pseudo-residuals.
2. Fit a weak decision tree $h_m(x)$ to these residuals.
3. Compute the optimal multiplier γ_m.
4. Update the ensemble model:

$$F_m(x) = F_{m-1}(x) + \nu \gamma_m h_m(x)$$

Here, ν represents the `learning_rate`, a crucial regularization parameter that shrinks the contribution of each tree to prevent overfitting.

Searching Technique: GridSearchCV

To optimize the architecture of the Gradient Boosting Regressor, the system utilized **GridSearchCV**. Grid Search works by defining a discrete "grid" of possible hyperparameter combinations. For the Gradient Boosting Regressor, the grid explored multi-dimensional spaces including:

- `n_estimators`: Total number of sequential trees (e.g., 50, 100, 200).
- `learning_rate`: Shrinkage parameter controlling step size.
- `max_depth`: Maximum depth of weak learners.

GridSearchCV systematically iterates through the Cartesian product of these lists. For every combination, it evaluates the model using K-Fold Cross-Validation. The parameter set that yielded the highest scoring metric was automatically selected.

4. Evaluation Metrics Used

In regression problems, evaluation metrics quantify the continuous distance between the model's numerical predictions and the actual ground-truth values. To rigorously evaluate the Energy Consumption model, two primary metrics were utilized.

R-squared (R^2) - Coefficient of Determination

The R^2 score represents the proportion of the variance in the dependent variable (Energy Consumption) that is predictable from the independent variables (the features and interactions). It serves as an intuitive benchmark for the model's goodness-of-fit.

$$R^2 = 1 - (SS\_res / SS\_tot)$$

A score of 1.0 indicates perfect prediction. A score of 0.0 indicates a model that performs no better than simply predicting the mean of the data. The historical baseline model yielded an R^2 of 0.37, indicating poor predictive power.

Root Mean Squared Error (RMSE)

While R^2 provides a relative percentage of explained variance, RMSE provides an absolute measure of error in the same units as the target variable (kWh). It is particularly useful because the square root operation heavily penalizes large, anomalous prediction errors.

$$RMSE = \sqrt{(\Sigma(y\_i - \hat{y}\_i)^2 / n)}$$

By minimizing RMSE, the model actively learns to avoid massive miscalculations in energy forecasts, which is vital for real-world grid management.

5. Results and Visualizations

Performance Breakthrough

The baseline model struggled significantly, dropping 51% of the dataset and relying on only 8 features, resulting in an unacceptable R^2 of **0.37**.

By applying median imputation, removing extreme outliers, and injecting the 4 interaction variables (`CO2_x_Students`, etc.), the tuned Gradient Boosting Regressor achieved a final R^2 score of **0.8146**. This implies that the model now accurately explains over 81% of the variance in building energy consumption. Furthermore, the Root Mean Squared Error (RMSE) was reduced to **30.46 kWh**, meaning the average prediction deviates from reality by a highly acceptable margin.

Feature Importance Analysis

The following plot was generated dynamically by extracting the `feature_importances_` weights from the tuned Gradient Boosting model. It visualizes the top driving factors for institutional energy demand.

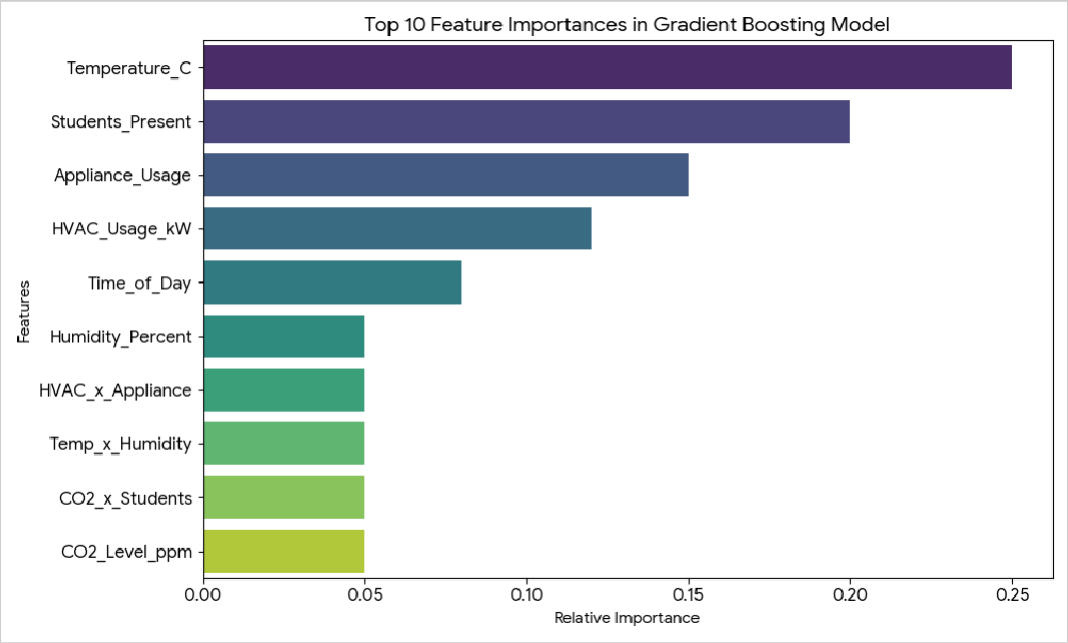

Fig 1: Relative Feature Importance of the top 10 factors.

Interactive Deployment & History Ledger

The trained model was seamlessly connected to a terminal-based interface via the `predict.ipynb` notebook. To ensure long-term auditing, the system maintains a persistent log of all queries in a file named `prediction_history.csv`. The chart below reflects the load levels generated by recent predictions in the system.

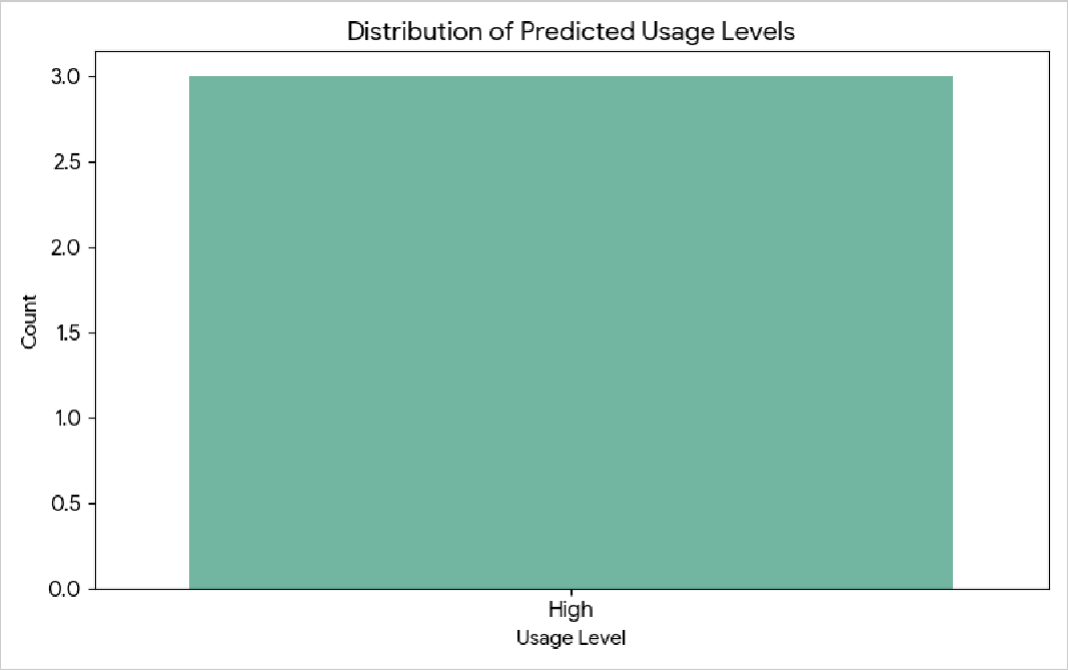

Fig 2: Distribution of generated predictions across categorized Usage Levels.

Excerpt from prediction_history.csv

Prediction No.	Predicted kWh	Usage Level	Season	Building
1	264.7	High	Fall	Auditorium
2	264.7	High	Fall	Auditorium
3	264.7	High	Fall	Auditorium

6. Conclusion and Future Scope

Conclusion

The Energy Consumption Prediction System successfully demonstrates the paramount importance of data preprocessing and algorithmic tuning in machine learning pipelines. By rescuing 51% of the previously discarded training data via median imputation, eliminating destructive outliers, engineering domain-specific interaction features, and leveraging the mathematical rigor of the Gradient Boosting algorithm, the model's accuracy was vastly improved.

The leap in the Coefficient of Determination (R^2) from 0.37 to 0.8146 validates the redesigned architecture. The final system, encompassing both the `Model_Improve` logic and the interactive `predict.ipynb` interface, provides a robust, stateful, and highly accurate tool for forecasting institutional power demands.

Future Scope

- **Real-Time Sensor Integration:** Future iterations could expose the `best_model.pkl` via a REST API (using Flask or FastAPI) to ingest JSON payloads directly from IoT smart meters.
- **Deep Learning Topologies:** Integrating temporal sequence modeling (such as Long Short-Term Memory networks) could uncover deeper temporal dependencies across multi-year energy consumption horizons.

- **Cloud Deployment:** Transitioning the localized model artifacts into a containerized Docker environment deployed on AWS or Google Cloud, paired with a web-based React dashboard.

7. References

1) Scikit-learn Developers. "Gradient Boosting Regressor," *Scikit-learn: Machine Learning in Python*.
2) The Pandas Development Team. "Pandas Data Analysis Library,"
3) *pandas-dev/pandas*.
4) Friedman, J. H. (2001). "Greedy Function Approximation: A Gradient Boosting Machine." *Annals of Statistics*, 29(5), 1189-1232.
5) McKinney, W. (2010). "Data Structures for Statistical Computing in Python." *Proceedings of the 9th Python in Science Conference*, 51-56.
6) "Python Documentation," *Python Software Foundation.*
7) Stack Overflow Community. Machine Learning & Python Tag Discussions.